

Claude Code LLM Interaction Log & Summary

Project Topic: Godot 4 Reinforcement Learning (Tabular Expected SARSA) **Framework/Environment:** Godot + Python TCP Bridge

Overview & High-Level Summary

This document systematically organizes all user Prompts submitted to Claude Code during the development of a **Tabular Expected SARSA** reinforcement learning agent in Godot 4 with Python, alongside the concise AI response summaries. It documents the entire progression from initial TCP architecture setup to achieving a 100% greedy evaluation success rate.

#	User Core Prompt Intent	AI Key Action & Technical Resolution Summary
1	Setup TCP architecture & Tabular Expected SARSA for Godot scene	Implemented `tcp_bridge.gd`, `rl_environment.gd`, and Python agent/server/training scripts.
2	Set episodes to 3,000 & adjust bridge crossing physics logic	Corrected Marker2D position to (670, 505) and enhanced Y-axis tolerance for bridge navigation.
3	Enable ceiling bump detection & guarantee eating 2 apples per episode	Added `hit_ceiling` state feature, `-5.0` penalty, and created `episode_runner.py`/`evaluate.py`.
4	Export chat history & improve policy exploration for double apples	Applied optimistic Q-initialization ($Q_{init} = 5.0$) to boost exploration; generated `chat_export.md`.
5	Accelerate training speed & resolve freezing/tie-break issues	Enabled `--headless --fixed-fps 480` (25-30x speedup), added checkpointing and tie-break randomness.
6	Diagnose infinite loops near enemies & fix non-determinism	Lowered danger penalty from -1.0 to -0.3; moved physics calculations to `_physics_process`.
7	Analyze state space vs. adding velocity dimensions (vel_x, vel_y)	Evaluated spatial explosion (1.2k to 10.5k states); decided to retain 10D compact state representation.
8	Implement Best Checkpoint saving & verify 100% convergence rate	Added `best_q_table.json` saving logic; captured 100% greedy success policy at episode 2600.

Detailed Prompts and AI Response Logs

Prompt #1: Architecture Setup & Protocol Communication

I want to use Python + TCP to implement Expected SARSA (do not use embedded GDScript). Add `rl\_environment.gd` and `tcp\_bridge.gd` on the Godot side. Scene: player spawns at top-right, needs to collect 2 apples in different positions, dodge 2 snails, and use a bridge connecting top and bottom layers. Route: collect bottom apple first, cross bridge while dodging snails, then collect top apple. Use tabular Expected SARSA... Set num_episodes = 3000.

AI Response Summary & Solution

- **Godot Side:** Created `tcp\_bridge.gd` (TCP client connecting to 127.0.0.1:5555) and `rl\_environment.gd` (environment wrapper discretization into a 9-element state vector and 6 discrete actions).
- **Python Side:** Built `expected\_sarsa\_agent.py` (Strict Expected SARSA update rule), `tcp\_server.py`, and `train.py`.
- **Update Rule:** $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \sum_{a'} \pi(a'|s')Q(s',a') - Q(s,a)]$

Prompt #2: Bridge Positioning & Navigation Calibration

The character should eat the bottom apple first, then move to the bridge above in the middle to dodge snails, rather than running underneath the bridge and jumping up towards the top-left apple.

AI Response Summary & Solution

Identified height estimation errors in the former implicit bridge coordinates.

- Added a dedicated `Bridge` Marker2D node in `level_root.tscn` at exact coordinates (670, 505).
- Updated bridge detection logic in `rl_environment.gd`: added Y-axis tolerance `BRIDGE_VERTICAL_TOLERANCE = 70` to avoid false positives when walking under the bridge.

Prompt #3: Ceiling Bump Handling & Consistent Dual-Apple Target

Due to assignment requirements, the agent must collect both apples in every single episode. If it hits its head on an overhead obstacle in the middle, it should try moving right and jumping up again. Make sure it can reliably eat two apples per run.

AI Response Summary & Solution

- **State Vector Expansion:** Added a 10th state dimension `hit_ceiling` using Godot's `is_on_ceiling()`.
- **Penalty Mechanism:** Introduced `CEILING_BUMP_PENALTY = -5.0`.
- **Evaluation Scripts:** Created `episode_runner.py` and `evaluate.py` for testing policy performance under full greedy mode ($\epsilon = 0$).

Prompt #4: Exploration Enhancement & Chat Log Export

Export our conversation history. How should we modify the strategy so that it eats both apples every single run without hitting the middle snails? You decide the best approach.

AI Response Summary & Solution

- Avoided hardcoded rule-based hacks, maintaining a pure reinforcement learning algorithmic approach.
- **Optimistic Q-Initialization:** Set default Q-table initial values to `Q_init = 5.0` to encourage early state-action exploration.
- **Epsilon Decay:** Reduced minimum exploration rate from 0.05 to 0.02.
- Exported full chat log to `chat_export.md`.

Prompt #5: Training Acceleration & Resumable Checkpoints

Can you speed up the training process? 3,000 episodes are taking too long to finish. What is the current progress?

AI Response Summary & Solution

- **Headless Mode Acceleration:** Configured Godot execution flags `godot --headless --fixed-fps 480`, achieving a 25-30x speedup (3,000 episodes complete in ~10-15 min).
- **Checkpoint Resumption:** Added state saving and loading via `q_table.json` and `training_state.json` in `train.py`.
- **Tie-Breaking Fix:** Resolved a deadlock issue where equal Q-values defaulted to Action 0 (Idle) by enforcing random selection among top actions.

Prompt #6: Enemy Hazard Deadlock Diagnosis & Determinism Fix

Analyze why the policy gets stuck oscillating near snails and times out. Also fix non-deterministic behavior in the environment.

AI Response Summary & Solution

- **Reward Rebalancing:** Discovered that `DANGER_ZONE_PENALTY = -1.0` canceled out distance potential rewards (+1.0), causing oscillation. Reduced danger penalty to `-0.3`.
- **Determinism Alignment:** Moved all snail movements and timer updates from `_process` (render frame) to `_physics_process` (fixed physics frame) to remove frame-rate jitter.

Prompt #7: State Space Complexity Analysis (Velocity Dimension)

Analyze whether we should include velocity dimensions (`vel_x` and `vel_y`) in the state discretization.

AI Response Summary & Solution

Evaluated impact: Adding 3x3 velocity bins increases discrete state space from ~11.6k to over 105k states. Given the 3,000 episode training budget, data sparsity degraded policy quality. Consequently retained the compact 10-dimensional state representation.

Prompt #8: Best Checkpoint Saving & 100% Greedy Convergence Verification

The agent succeeded on its first try! My friend mentioned theirs succeeds 100% of the time, and I need to achieve the same. Add a best checkpoint feature. Run the AI to control Godot.

AI Response Summary & Solution

- Added evaluation routine in `train.py` saving `best_q_table.json` whenever a greedy evaluation run ($\epsilon = 0$) successfully collects 2 apples and reaches the goal.
- **Final Verification:** Successfully captured optimal Q-table at episode 2600. Running `evaluate.py` confirmed a **100% success rate across all test runs (2 apples collected, ~65-175 steps per episode)** in both Headless and Windowed visual modes.